

TEAM=13

MEMBERS :

- ROHIT RANJAN
- ROHIT KUMAR MEENA
- DAVINDARPAL SINGH
- SUMIT SARKAR
- ANSHUL GAHARWAL

SCHEDULING RULE EVALUATOR



PROBLEM STATEMENT :

Develop a Python program to evaluate various job sequencing rules for a single-machine environment with n jobs. Additionally, implement a graphical user interface (GUI) that provides an interactive tool for users to explore different sequencing rules and visualize their impacts on job scheduling outcomes in real time.

- First-Come, First-Served (FCFS)
- Shortest Processing Time (SPT)
- Longest Processing Time (LPT)
- Smallest Slack Time
- Smallest Criticality Ratio
- Random Rule (User-Specified)



LIBRARY/MODULE USED:



TKINTER



Provides tools for building the graphical user interface (GUI).

Used for user input, buttons, and displaying output.

RANDOM

RANDOM

Generates random processing times and due dates for jobs.

Provides the option to shuffle job orders for the Random rule

MATPLOTLIB



Plots bar charts of performance measures (lateness, flow time, utilization).

Enhances visual analysis of results across different scheduling rules.



SCHEDULING RULES:

FIRST COME, FIRST SERVE

Jobs are processed in the order they arrive, with no priority adjustments.

LONGEST PROCESSING TIME

Jobs with the shortest completion times are prioritized to minimize overall job time.

SHORTEST PROCESSING TIME

Jobs with the longest completion times are processed first to optimize resource utilization.

MINIMUM SLACK

Jobs with the least slack (time until due date minus remaining processing time) are prioritized to avoid delays.

LEAST CRITICALITY RATIO

Jobs are prioritized by the ratio of time remaining until due date to processing time, emphasizing time-sensitive tasks.

RANDOM

Jobs are assigned with no predetermined order, creating a non-deterministic sequence.



USER DEFINED FUNCTIONS

calculate_measures (rule,
processing_times, due_dates,
user_order=None)

- Calculates performance measures: average lateness, flow time, utilization, and jobs in the system.
- Sorts jobs based on the selected rule (e.g., FCFS, SPT, etc.).

get_user_order(n)

- Allows the user to input a custom job sequence for the Random rule.
- Validates and returns a job order based on user input.

plot_results(results)

- Plots bar charts for Average Lateness, Flow Time, and Utilization for each rule.
- Highlights the best-performing rule in each chart for easy comparison.

display_results()

- Collects input, generates job data, and computes results for all rules.
- Displays best rules based on each performance measure and outputs metrics for each rule.
- Triggers plot generation to visualize rule comparisons.



```
loads > te_measure.  
r as tk  
import simpledialog, messagebox  
  
tlib.pyplot as plt  
  
_measures(rule, processing_times, due_date  
rocessing_times)  
bs = list(range(n))  
  
= "FCFS":  
# FCFS doesn't require any sorting  
== "SPT":  
d_jobs.sort(key=lambda x: processing_times  
== "LPT":  
d_jobs.sort(key=lambda x: -processing_time  
== "Smallest Slack":  
d_jobs.sort(key=lambda x: due_dates[x] - p  
== "Smallest Criticality Ratio":  
d_jobs.sort(key=lambda x: due_dates[x] / p  
== "Random":  
er_order:  
orted_jobs = user_order  
  
andom.shuffle(sorted_jobs)  
  
= 0  
sum = 0  
e_flow_time = 0  
rocessing_times = []  
e_dates = []  
n sorted_jobs:  
d_processing_times.append(processing_times  
d_due_dates.append(due_dates[job])  
time += processing_times[job]  
ative_flow_time += flow_time  
.../0... flow_time... due_dates[job]
```

GUI INTERFACE

Input Fields and Validation

User Inputs: Fields for Number of Jobs, Processing Time Case, and Due Date Case.
Validation: Ensures valid job entries; prompts user errors for invalid values or zero jobs.

Calculate Button

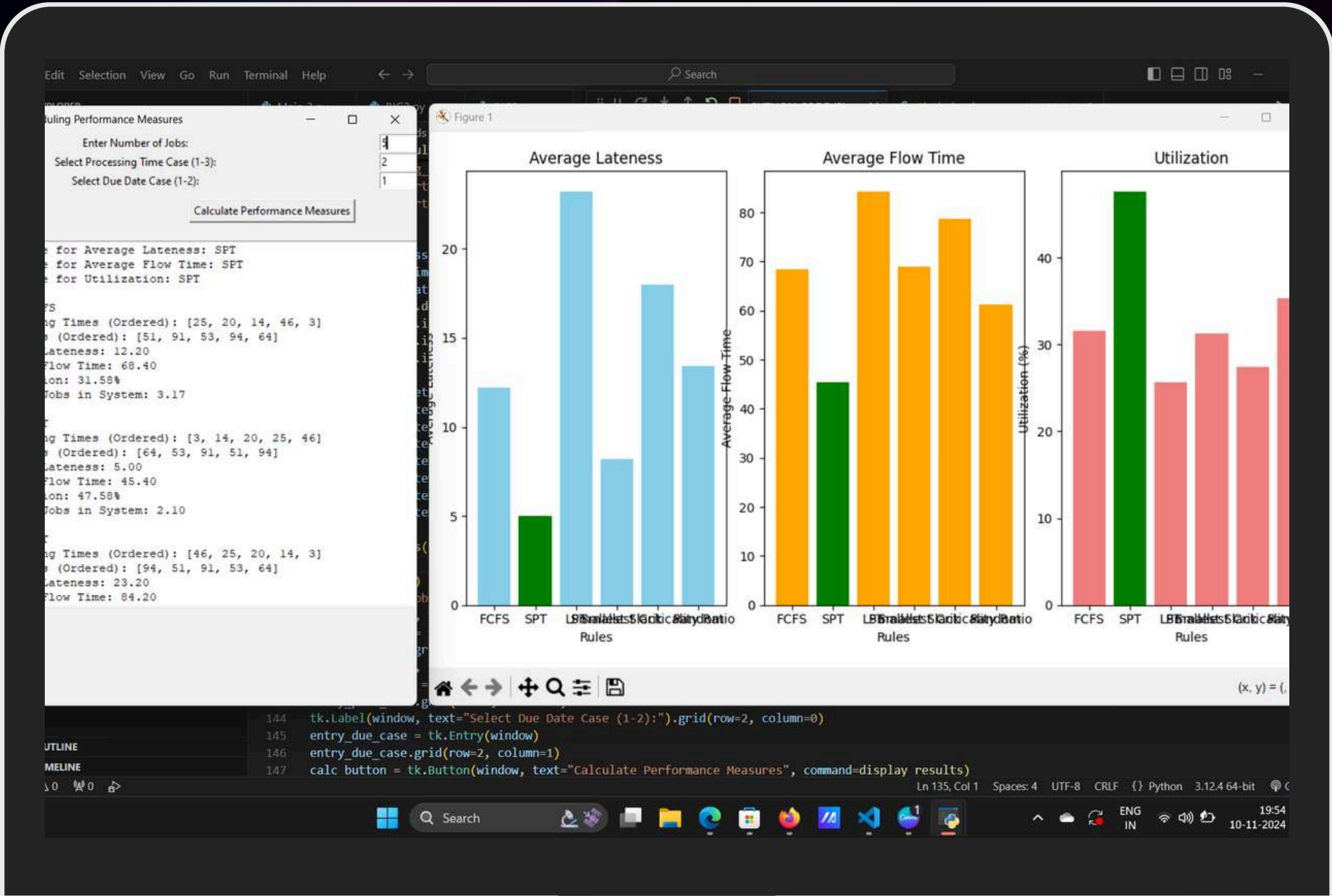
Functionality: Triggers `display_results` to apply the rules and compute performance.
Custom Order: Prompts for custom job order in “Random” rule, with format validation for accurate processing.

Results Display

Text Output: Organized metrics for each rule — Average Lateness, Flow Time, Utilization — enabling quick comparison.

Performance Charts

Graphical Insights: Matplotlib bar charts for lateness, flow time, and utilization with best metrics highlighted, aiding in optimal rule selection. ➡



THANK YOU!



```
def calculate_measures(rule, processing_times, due_dates, user_order=None):
    if user_order:
        sorted_jobs = user_order
    else:
        sorted_jobs = random.shuffle(sorted_jobs)

    flow_time = 0
    lateness_sum = 0
    cumulative_flow_time = 0
    sorted_processing_times = []
    sorted_due_dates = []
    for job in sorted_jobs:
        sorted_processing_times.append(processing_times[job])
        sorted_due_dates.append(due_dates[job])
        flow_time += processing_times[job]
        cumulative_flow_time += flow_time
        lateness = max(0, flow_time - due_dates[job])
        lateness_sum += lateness

    avg_lateness = lateness_sum / n
    avg_flow_time = cumulative_flow_time / n
    utilization = 100 * sum(processing_times) / cumulative_flow_time
    avg_jobs_in_system = cumulative_flow_time / sum(processing_times)

    return avg_lateness, avg_flow_time, utilization, avg_jobs_in_system, sorted_processing_times, sorted_due_dates

def get_user_order(n):
    user_order_str = simpledialog.askstring("Input", f"Enter the order of job indices (0-{n-1}) separated by commas for the Random rule:")
    if user_order_str:
        try:
            return list(map(int, user_order_str.split(',')))
        except ValueError:
```

